

Machine Learning Applications

NumPy — Numerical Computing with Python

NUS Institute of Systems Science · Postgraduate Programme

Why NumPy?

NumPy (Numerical Python) is the foundation of the entire Python scientific computing stack. It provides fast, memory-efficient **multi-dimensional arrays** and vectorised operations — making it 10–100× faster than plain Python lists for numerical work.

NumPy vs Python Lists

Feature	Python List	NumPy Array
Speed	Slow (loops)	Fast (vectorised)
Memory	High overhead	Compact, contiguous
Type	Mixed types	Homogeneous dtype
Operations	Element-by-element loops	Broadcast operations
ML Libraries	Not compatible	Scikit-learn, TensorFlow, PyTorch

💡 Every ML library uses NumPy arrays under the hood. Mastering NumPy is essential before learning Pandas, Scikit-learn, or deep learning.

```
why_numpy.py ▶ Run

import numpy as np

# Speed comparison: sum 1 million numbers
import time

# Python list
py_list = list(range(1_000_000))
start = time.time()
total = sum(py_list)
py_time = time.time() - start

# NumPy array
np_arr = np.arange(1_000_000)
start = time.time()
total = np.sum(np_arr)
np_time = time.time() - start

OUTPUT
```

Part 1

Creating Arrays — Dimensions, Shape & dtype

Creating NumPy Arrays

The core object is the **ndarray** (N-dimensional array). You can create arrays from Python lists/tuples using `np.array()`, or use built-in constructors like `np.zeros()`, `np.ones()`, `np.arange()`, and `np.linspace()`.

Common Constructors

- ▶ `np.array([1,2,3])` — from list or tuple
- ▶ `np.zeros((2,3))` — all zeros
- ▶ `np.ones((3,3))` — all ones
- ▶ `np.arange(0,10,2)` — like `range()`, returns array
- ▶ `np.linspace(0,1,5)` — evenly spaced floats
- ▶ `np.eye(3)` — 3×3 identity matrix
- ▶ `np.full((2,3), 7)` — fill with a value

💡 `np.linspace` is especially useful in ML for generating evenly spaced values for plotting decision boundaries or learning rate schedules.

creating_arrays.py ▶ Run

```
import numpy as np

# From list
a = np.array([1, 2, 3])
print("From list:", a)

# From tuple
b = np.array((4, 5, 6))
print("From tuple:", b)

# 2D array (matrix)
mat = np.array([[1, 2, 3], [4, 5, 6]])
print(f"\n2D array:\n{mat}")
print(f"Type: {type(mat)}")

# Built-in constructors
```

OUTPUT

Dimensions, Shape & dtype

Every array has a **ndim** (number of dimensions), **shape** (size along each dimension), **size** (total elements), and **dtype** (data type). Understanding these properties is critical for debugging shape mismatches in ML.

Key Attributes

- ▶ **.ndim** — number of axes (0D=scalar, 1D=vector, 2D=matrix)
- ▶ **.shape** — tuple of sizes, e.g. (3,4) = 3 rows × 4 cols
- ▶ **.size** — total element count
- ▶ **.dtype** — data type (int64, float64, bool, etc.)

💡 Shape errors are the #1 debugging headache in ML. Always check **.shape** when things go wrong! A (3,) array and a (3,1) array behave very differently.

```
shape_dtype.py ▶ Run

import numpy as np

# 0D (scalar)
s = np.array(42)
print(f"0D: value={s}, ndim={s.ndim}, shape={s.shape}")

# 1D (vector)
v = np.array([1, 2, 3, 4, 5])
print(f"1D: ndim={v.ndim}, shape={v.shape}, size={v.size}")

# 2D (matrix) — 2 rows × 3 columns
m = np.array([[1, 2, 3], [4, 5, 6]])
print(f"2D: ndim={m.ndim}, shape={m.shape}, size={m.size}")

# 3D — 2 "layers" of 2×3 matrices
t = np.array([[[1, 2, 3], [4, 5, 6]],
```

OUTPUT

Reshaping Arrays

Reshaping changes the shape without changing the data. The total number of elements must stay the same. Use -1 to let NumPy calculate one dimension automatically. `flatten()` converts any array to 1D.

Reshaping Rules

- ▶ `arr.reshape(rows, cols)` — new shape, same data
- ▶ Product of dimensions must equal `arr.size`
- ▶ `reshape(-1, 3)` — auto-calculate rows
- ▶ `reshape(4, -1)` — auto-calculate cols
- ▶ `arr.flatten()` — always returns 1D copy
- ▶ `arr.ravel()` — 1D view (no copy, faster)
- ▶ `arr.T` — transpose (swap rows ↔ cols)

💡 In ML, you constantly reshape data: flattening images to vectors, reshaping model inputs, transposing weight matrices. The -1 trick is used everywhere.

```
reshaping.py ▶ Run

import numpy as np

arr = np.arange(1, 13) # [1, 2, ..., 12]
print(f"Original: {arr}, shape={arr.shape}")

# Reshape to 4x3
m = arr.reshape(4, 3)
print(f"\nReshaped (4,3):\n{m}")

# Reshape to 2x6 using -1
m2 = arr.reshape(2, -1) # -1 = auto
print(f"\nReshaped (2,-1):\n{m2}")

# Reshape to 3D: 2 layers x 2 rows x 3 cols
m3 = arr.reshape(2, 2, 3)
print(f"\n3D shape: {m3.shape}")
```

OUTPUT

Part 2

Indexing, Slicing & Boolean Selection

Indexing & Slicing

NumPy uses **0-based indexing**, just like Python lists. For multi-dimensional arrays, use comma-separated indices: `arr[row, col]`. Slicing uses the same `[start:stop:step]` syntax, applied per dimension.

Indexing Patterns

- ▶ `arr[0]` — first element (1D) or first row (2D)
- ▶ `arr[1, 2]` — row 1, col 2 in a 2D array
- ▶ `arr[-1]` — last element/row
- ▶ `arr[0:3]` — first 3 elements
- ▶ `arr[:, 0]` — entire first column
- ▶ `arr[0:2, 1:3]` — sub-matrix

💡 The comma syntax `arr[row, col]` is unique to NumPy — Python lists require `arr[row][col]`. NumPy's syntax is cleaner and faster.

```
indexing.py ▶ Run

import numpy as np

# 1D indexing & slicing
arr = np.array([10, 20, 30, 40, 50, 60, 70])
print(f"arr[0]   = {arr[0]}")
print(f"arr[-1]  = {arr[-1]}")
print(f"arr[1:5]  = {arr[1:5]}")
print(f"arr[:,2]  = {arr[:,2]}")

# 2D indexing
mat = np.array([[1,2,3], [4,5,6], [7,8,9]])
print(f"\nMatrix:\n{mat}")
print(f"mat[0,0]   = {mat[0,0]}")
print(f"mat[1,2]   = {mat[1,2]}")
print(f"mat[-1,-1] = {mat[-1,-1]}")
```

OUTPUT

Boolean Indexing (Filtering)

Applying a condition to an array creates a **boolean mask** (True/False array). Passing this mask back into the array extracts only the elements where the mask is True. This is how you filter data without loops.

How It Works

- ▶ **Step 1:** Apply condition → boolean array
- ▶ **Step 2:** Use boolean array as index → filtered result
- ▶ Combine conditions with & (and), | (or), ~ (not)
- ▶ `np.where(condition, x, y)` — conditional replacement

💡 Boolean indexing replaces loops for data filtering. In ML, you'll use it to select features, filter outliers, and mask missing values — all without writing a single for-loop.

```
boolean_indexing.py ▶ Run

import numpy as np

mat = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(f"Matrix:\n{mat}")

# Step 1: Create boolean mask
mask = mat % 2 == 0
print(f"\nEven mask:\n{mask}")

# Step 2: Apply mask to extract values
evens = mat[mask]
print(f"Even values: {evens}")

# One-liner (common pattern!)
print(f"Values > 5: {mat[mat > 5]}")
```

OUTPUT

Part 3

Array Operations — Vectorisation, Broadcasting & Linear Algebra

Element-Wise Operations

Arithmetic operators (+, -, *, /, **) work **element-wise** on NumPy arrays — no loops needed. This is called **vectorisation**. When operating a scalar with an array, the scalar is applied to every element (**broadcasting**).

Two Flavours

- ▶ **Scalar + Array:** `arr + 2` adds 2 to every element
- ▶ **Array + Array:** `a + b` adds element-by-element (same shape required)
- ▶ `*` is element-wise, NOT matrix multiply!
- ▶ For matrix multiplication use `np.dot(a, b)` or `a @ b`

💡 Vectorised code is not just cleaner — it's orders of magnitude faster. Never write `for i in range(len(arr))` when a NumPy operation exists.

```
elementwise.py ▶ Run

import numpy as np

a = np.array([[1, 0], [1, 2]])
print(f"a = \n{a}")

# Scalar operations
print(f"\na + 2 = \n{a + 2}")
print(f"a * 3 = \n{a * 3}")
print(f"a ** 2 = \n{a ** 2}")

# Array + Array (element-wise)
b = np.array([[0, 2], [1, 1]])
print(f"\nb = \n{b}")
print(f"a + b = \n{a + b}")
print(f"a * b = \n{a * b}") # NOT matrix multiply!

OUTPUT
```

Aggregation & Statistics

NumPy provides fast aggregation functions: **sum**, **mean**, **std**, **min**, **max**, **argmin**, **argmax**. The **axis** parameter controls the direction: **axis=0** operates down rows (column-wise), **axis=1** operates across columns (row-wise).

Axes Explained

- ▶ **axis=None** — operate on ALL elements (default)
- ▶ **axis=0** — collapse rows → result per column
- ▶ **axis=1** — collapse columns → result per row
- ▶ **argmin/argmax** — index of min/max value

Quick Think

For a (3,4) array, what shape does `np.sum(arr, axis=0)` return?

```
aggregation.py ▶ Run

import numpy as np

data = np.array([[10, 20, 30],
                 [40, 50, 60]])
print(f"Data:\n{data}\n")

# Global aggregation
print(f"Sum (all): {np.sum(data)}")
print(f"Mean (all): {np.mean(data)}")
print(f"Std (all): {np.std(data):.2f}")

# axis=0: collapse rows → per-column result
print(f"\nSum axis=0 (per col): {np.sum(data, axis=0)}")
print(f"Mean axis=0: {np.mean(data, axis=0)}")

# axis=1: collapse cols → per-row result
print(f"\nSum axis=1 (per row): {np.sum(data, axis=1)}")

OUTPUT
```

Matrix Multiplication (Dot Product)

Matrix multiplication uses `np.dot(a, b)` or the `@` operator. The rule: **$(m \times n) @ (n \times p) = (m \times p)$** . The inner dimensions must match. This is fundamentally different from `*` (element-wise).

* vs @ vs np.dot()

Operation	Syntax	Meaning
Element-wise	<code>a * b</code>	Multiply position by position
Matrix multiply	<code>a @ b</code>	Linear algebra dot product
Matrix multiply	<code>np.dot(a, b)</code>	Same as <code>@</code>

💡 Matrix multiplication is at the heart of ML: neural network forward passes are just chains of matrix multiplications. Understanding shapes here prevents many bugs later.

```
matrix_multiply.py ▶ Run

import numpy as np

# (1x2) @ (2x1) = (1x1) - dot product
a = np.array([1, 2])          # shape (2,)
b = np.array([[3], [4]])      # shape (2,1)
print(f"a shape: {a.shape}")
print(f"b shape: {b.shape}")
print(f"a @ b = {a @ b}")     # [11]

# (2x3) @ (3x2) = (2x2)
A = np.array([[1,2,3],[4,5,6]])
B = np.array([[1,0],[0,1],[1,1]])
print(f"\nA (2x3):\n{A}")
print(f"B (3x2):\n{B}")
print(f"A @ B (2x2):\n{A @ B}")

OUTPUT
```

Part 4

Stacking, Splitting, Iteration & Searching

Stacking & Concatenation

Concatenation joins arrays along an existing axis. **Stacking** joins arrays along a new axis. `np.vstack` stacks vertically (row-wise), `np.hstack` stacks horizontally (column-wise).

Key Functions

- ▶ `np.concatenate([a,b], axis=0)` — join along existing axis
- ▶ `np.vstack([a,b])` — stack vertically (add rows)
- ▶ `np.hstack([a,b])` — stack horizontally (add cols)
- ▶ `np.stack([a,b], axis=0)` — stack along NEW axis
- ▶ `np.array_split(arr, n)` — split into n parts

💡 In ML, you stack feature vectors into matrices, concatenate batches of data, and split datasets into train/test sets using these operations.

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
c = np.array([7, 8, 9])

# vstack - stack as rows
v = np.vstack([a, b, c])
print(f"vstack:\n{v}  shape={v.shape}")

# hstack - concatenate side by side
h = np.hstack([a, b, c])
print(f"\nhstack: {h}  shape={h.shape}")

# concatenate along axis=0 (same as vstack for 2D)
m1 = np.array([[1,2],[3,4]])
```

OUTPUT

Iteration, Search & Sort

You can iterate arrays with for loops, but prefer vectorised operations when possible. `np.where()` finds indices matching a condition, and `np.sort()` returns a sorted copy.

Key Functions

- ▶ `for x in arr:` — iterate over first axis
- ▶ `np.nditer(arr)` — iterate over all elements
- ▶ `np.where(condition)` — returns indices of True
- ▶ `np.sort(arr)` — sorted copy
- ▶ `np.argsort(arr)` — indices that would sort
- ▶ `np.unique(arr)` — unique values

💡 In practice, avoid explicit loops on NumPy arrays — they negate the speed advantage. Use boolean indexing and vectorised functions instead.

search_sort.py ▶ Run

```
import numpy as np

# Iteration (2D: iterates over rows)
mat = np.array([[1,2,3],[4,5,6]])
print("Iterating 2D (rows):")
for row in mat:
    print(f" {row}")

# Iterate all elements
print("\nAll elements:", end=" ")
for x in np.nditer(mat):
    print(x, end=" ")

# Search with np.where
arr = np.array([1, 2, 3, 4, 5, 4, 4])
idx = np.where(arr == 4)
```

OUTPUT

Part 5

Random Number Generation & Practical Applications

Random Number Generation

`numpy.random` provides functions for generating random numbers from various distributions. Use `np.random.seed()` for **reproducibility** — essential in ML experiments.

Key Functions

- ▶ `random.randint(low, high, size)` — random integers
- ▶ `random.rand(d0, d1)` — uniform $[0, 1)$
- ▶ `random.randn(d0, d1)` — standard normal
- ▶ `random.choice(arr, size)` — random sampling
- ▶ `random.shuffle(arr)` — shuffle in place
- ▶ `random.seed(42)` — fix the random state

💡 **Always set a seed** when running ML experiments. This makes your results reproducible — a requirement for scientific work and debugging.

```
random_functions.py ▶ Run

import numpy as np

# Set seed for reproducibility
np.random.seed(42)

# Random integers
print("randint:", np.random.randint(0, 100, size=5))

# Uniform [0, 1)
print("rand:", np.random.rand(3).round(3))

# Standard normal (mean=0, std=1)
print("randn:", np.random.randn(5).round(3))

# Random choice from array
fruits = ["apple", "banana", "cherry"]

OUTPUT
```

Broadcasting

Broadcasting is NumPy's mechanism for operating on arrays of different shapes. NumPy automatically expands the smaller array to match the larger one — no copies made, very efficient. Rules: dimensions are compared right-to-left; they're compatible if equal or one of them is 1.

Broadcasting Rules

- ▶ Compare shapes from the trailing (rightmost) dimension
- ▶ Dimensions are compatible if they are **equal** or one is **1**
- ▶ Missing dimensions are treated as size 1
- ▶ Example: $(3,4) + (4,) \rightarrow (3,4) + (1,4) \rightarrow$ works!
- ▶ Example: $(3,4) + (3,1) \rightarrow$ works!
- ▶ Example: $(3,4) + (3,) \rightarrow$ ERROR!

💡 Broadcasting eliminates the need for explicit loops and temporary copies. In ML, it's how bias is added to every sample in a batch, or how normalisation is applied column-wise.

```
broadcasting.py ▶ Run

import numpy as np

# Scalar broadcast: (2,3) + scalar
m = np.array([[1,2,3],[4,5,6]])
print(f"m + 10:\n{m + 10}\n")

# 1D broadcast: (2,3) + (3,) → adds to each row
row = np.array([100, 200, 300])
print(f"m + [100,200,300]:\n{m + row}\n")

# Column broadcast: (2,3) + (2,1) → adds to each col
col = np.array([[10], [20]])
print(f"m + [[10],[20]]:\n{m + col}\n")

# Practical ML example: normalise columns

OUTPUT
```

Summary & Next Steps

NumPy is the foundation of scientific Python. Every ML library — Pandas, Scikit-learn, TensorFlow, PyTorch — is built on NumPy arrays. Next session: **Pandas** for data analysis.

What We Covered

- ▶ Why NumPy: speed, memory efficiency, vectorisation
- ▶ Creating arrays: `np.array`, `zeros`, `ones`, `arange`, `linspace`
- ▶ Dimensions, shape, dtype — understanding array properties
- ▶ Reshaping, flattening, transposing
- ▶ Indexing, slicing, and boolean filtering
- ▶ Element-wise operations and matrix multiplication
- ▶ Aggregation functions with axis control
- ▶ Stacking, splitting, searching, sorting
- ▶ Random number generation and reproducibility
- ▶ Broadcasting — operating on mismatched shapes

Practice Resources

- ▶ [NumPy: Absolute Basics for Beginners](#)
- ▶ [Applied Introduction to NumPy](#)
- ▶ [NumPy Quickstart Tutorial \(Official\)](#)

💡 **Key practice areas:** reshaping arrays, boolean indexing, and understanding axis behaviour in aggregation functions. These are the skills that transfer directly to Pandas and ML model building.

Thank You

Questions & Discussion

Machine Learning Applications · NUS Institute of Systems Science